

Lecture 5a

SelectionSort & HeapSort

Sorting

- **Sorting** is the process of placing elements in order
- If elements are objects, they are ordered by a particular data member
- There are many algorithms for sorting, each having its own advantages
- No sorting algorithm is better than every other sorting algorithm all the time
- Choosing the right sorting algorithm for a problem requires careful consideration

SelectionSort

- Idea: Find the largest number from the unsorted numbers, place it at the correct location.
- Array: 30 60 50 20 40
- $n = 5$
- $i = 4$: 30 40 50 20 60*
- $i = 3$: 30 40 20 50* 60*
- $i = 2$: 30 20 40* 50* 60*
- $i = 1$: 20 30* 40* 50* 60* (sorted)

SelectionSort (cont.)

Algorithm SelectionSort (S, n)

Input: sequence S of n elements

Output: sorted sequence S

```
for i = n-1 downto 1
  maxIndex = i // assume maxIndex has the max element
  for j = 0 to i-1
    // if element at j is larger, then it is the new max
    if S[j] > S[maxIndex]
      maxIndex = j

  // swap if maxIndex has changed
  if maxIndex != i
    swap (S[i], S[maxIndex])
```

SelectionSort Time Complexity

- By using the same analysis as the prefixAverages1 algorithm in Lecture 1 (the "for" loops of the 2 algorithms are similar), SelectionSort runs in $\Theta(n^2)$ time

HeapSort

Algorithm HeapSort (S , n)

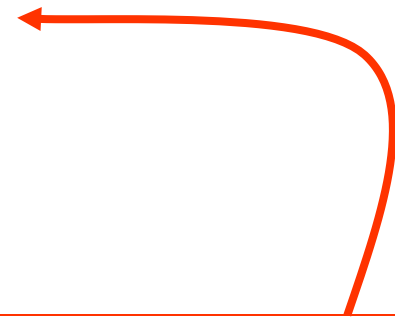
Input: sequence S of n elements

Output: sorted sequence S

$pq = \text{priority_queue}(S)$

for $i = n-1$ downto 1

$pq.\text{dequeue}(S[i])$



Create a heap-based priority queue from S

HeapSort (cont.)

Algorithm HeapSort (S , n)

Input: sequence S of n elements

Output: sorted sequence S

$pq = \text{priority_queue}(S)$

for $i = n-1$ downto 1

$pq.\text{dequeue}(S[i])$



In the loop, we start with the last sequence index

HeapSort (cont.)

Algorithm HeapSort (S , n)

Input: sequence S of n elements

Output: sorted sequence S

$pq = \text{priority_queue}(S)$

for $i = n-1$ downto 1

$pq.\text{dequeue}(S[i])$



On the first dequeue, the largest element from the heap will be placed into the last position of the sequence

HeapSort (cont.)

Algorithm HeapSort (S , n)

Input: sequence S of n elements

Output: sorted sequence S

$pq = \text{priority_queue}(S)$

for $i = n-1$ downto 1

$pq.\text{dequeue}(S[i])$



On the next dequeue (i is decremented), the remaining highest value from the heap will be placed in the next left sequence position

HeapSort (cont.)

Algorithm HeapSort (S , n)

Input: sequence S of n elements

Output: sorted sequence S

$pq = \text{priority queue}(S)$

for $i = n-1$ downto 1

$pq.\text{dequeue}(S[i])$



The loop stops when the index becomes less than 1

HeapSort (cont.)

- $S = 30 \ 60 \ 50 \ 20 \ 40$
- Form an Initial Heap
- $S/pq = 60 \ 40 \ 50 \ 20 \ 30$
- pq is a subset of S
- $i=4$: $S = 50 \ 40 \ 30 \ 20 \ 60^* \ (pq = 50 \ 40 \ 30 \ 20)$
- $i=3$: $S = 40 \ 20 \ 30 \ 50^* \ 60^* \ (pq = 40 \ 20 \ 30)$
- $i=2$: $S = 30 \ 20 \ 40^* \ 50^* \ 60^* \ (pq = 30 \ 20)$
- $i=1$: $S = 20 \ 30^* \ 40^* \ 50^* \ 60^* \ (pq = 20)$
- Sorted

HeapSort Time Complexity

- HeapSort runs in $\Theta(n \lg n)$ time on average
- It has a best-case time of $\Theta(n)$ if all elements have the same value
 - This is an unusual case, but we may want to consider it if many of the elements have the same value